Identificação do projeto

Título do projeto: Perfil das faixas populares nos charts da Deezer por gênero
Integrantes do grupo: Eduardo Fabri, João Correa, Rodrigo Alves e João Pedro Cardoso
Descrição breve do problema ou contexto: os charts da Deezer dizem quais faixas estão no topo de cada gênero, mas não o que elas têm em comum. O projeto coleta esses charts pela API pública da Deezer e prepara os dados para comparar, entre gêneros, a duração das faixas, a idade do lançamento, a popularidade do artista e a concentração de artistas no ranking.

Preencha este notebook ao longo dos dois RAs. Registre decisões e interpretações em poucas frases e acrescente as células de código necessárias em cada etapa.

Preparação do ambiente

Esta célula prepara as bibliotecas, as pastas do projeto e a origem dos dados. No Google Colab, ela clona o repositório para ter acesso ao retrato bruto da API já coletado.

```
# Preparação do ambiente: bibliotecas, pastas do projeto e origem dos dados
import subprocess
import sys
from pathlib import Path

import pandas as pd

URL_REPOSITORIO = "https://github.com/Jcliz/data-analytics.git"
RAMO = "main"  # no Colab, troque pelo ramo do repositório que quiser executar
NO_COLAB = "google.colab" in sys.modules

if NO_COLAB:
    if Path("data-analytics").exists():
        # Clone de uma execução anterior nesta sessão do Colab: traz a versão mais nova do ramo
        subprocess.run(["git", "-C", "data-analytics", "pull", "-q", "origin", RAMO], check=False)
    else:
        subprocess.run(["git", "clone", "-q", "-b", RAMO, URL_REPOSITORIO], check=True)

# A raiz é a pasta que contém o CONTEXT.md (funciona no Colab, na raiz ou dentro de notebook/)
candidatos = [Path("data-analytics"), Path.cwd(), Path.cwd().parent]
RAIZ = next((p.resolve() for p in candidatos if (p / "CONTEXT.md").exists()), None)
if RAIZ is None:
    raise FileNotFoundError(
        "Raiz do repositório não encontrada (pasta com o CONTEXT.md). Rode o notebook de dentro do clone "
        f"(raiz ou notebook/) ou, no Colab, confira se o ramo '{RAMO}' já tem a estrutura do projeto."
    )

PASTA_BRUTOS = RAIZ / "dados" / "brutos"
PASTA_PREPARADOS = RAIZ / "dados" / "preparados"
PASTA_STREAMING = RAIZ / "dados" / "streaming"
for pasta in (PASTA_BRUTOS, PASTA_PREPARADOS, PASTA_STREAMING):
    pasta.mkdir(parents=True, exist_ok=True)

# False: lê o retrato bruto versionado (ADR-0004). True: coleta de novo na API da Deezer.
BAIXAR_DA_API = False

pd.set_option("display.max_columns", 30)
pd.set_option("display.width", 120)
print(f"Raiz do projeto: {RAIZ}")
print(f"Ambiente: {'Google Colab' if NO_COLAB else 'local'} · Python {sys.version.split()[0]} · pandas {pd.__version__}")
```

RA1 — Data Ingestion, Cleaning and Initial Analysis

Nesta etapa, carregue, prepare e explore os dados do projeto.

1. Contexto e objetivo

Apresente brevemente o problema investigado e o que o grupo pretende descobrir com os dados.

Resposta do grupo:

Os charts da Deezer mostram as faixas mais ouvidas em cada gênero, mas não o que elas têm em comum. O grupo quer descrever o perfil dessas faixas e ver se ele muda entre gêneros. O recorte é um único retrato dos charts da Deezer, o chart geral e o de cada gênero, na data da coleta registrada na seção 2. A unidade de observação é a faixa. As perguntas são:

- **P1:** como a duração das faixas varia entre os gêneros;
- **P2:** que parte das faixas de cada gênero foi lançada nos 12 meses anteriores à coleta;
- **P3:** se faixas de artistas com mais fãs têm `rank` mais alto, no geral e dentro de cada gênero;
- **P4:** quantos artistas distintos ocupam cada chart de gênero e que fatia das posições fica com os 5 artistas com mais faixas.

O detalhe do recorte e das lacunas está na seção 01 do documento do RA1.

2. Fonte e ingestão dos dados

- Identifique a fonte dos dados, incluindo link ou referência de acesso.
- Descreva brevemente o dataset e o que seus registros representam.
- Explique como os dados foram carregados e implemente a ingestão.

Resposta do grupo:

A fonte é a API pública da Deezer (`https://api.deezer.com`, documentação em `https://developers.deezer.com/api`), acessada sem autenticação e sem chave. Coletamos os charts de faixas mais tocadas: o chart geral e um chart por gênero da lista `/genre`. Cada registro do `df_bruto` é um item de um chart, ou seja, uma faixa na posição em que apareceu em um chart de origem; a mesma faixa pode aparecer em mais de um chart, e essas repetições ficam no bruto. Cada item recebe os campos extras da faixa (`/track/{id}`, como `bpm`, `isrc` e `release_date`), do álbum (`/album/{id}`, como gênero, gravadora e data de lançamento) e do artista (`/artist/{id}`, como `nb_fan` e `nb_album`), juntados pelo id.

A ingestão é feita por código nas células abaixo: as chamadas passam por uma função com pausa entre requisições, novas tentativas e tratamento do objeto `error`, que a API devolve mesmo com HTTP 200. Os charts são paginados por `index` e `limit`. As respostas são gravadas cruas no retrato bruto `dados/brutos/AAAA-MM-DD/`, com um `metadados.json` da coleta, e o notebook lê esse retrato por padrão (`BAIXAR_DA_API = False`), para que os números sejam os mesmos em toda execução.

O retrato usado é o de 16/10/2026 (país aparente US): 6.591 chamadas, sem erros, 28 charts de 100 itens (o geral e 27 gêneros). O `df_bruto` tem 2.800 linhas e 102 colunas, com 2.581 faixas, 2.292 álbuns e 1.688 artistas únicos, todos com o detalhe juntado.

```
# Parâmetros da coleta e acesso à API
import json
import platform
import time
from datetime import datetime, timezone

import requests
from IPython.display import display

URL_API = "https://api.deezer.com"
LIMITE_PAGINA = 100       # itens por página do chart (parâmetro limit)
TETO_POR_CHART = 300      # máximo de itens lidos por chart, mesmo que a API ofereça mais
PAUSA_S = 0.15            # pausa entre chamadas; a API aceita cerca de 50 chamadas a cada 5 s
TENTATIVAS = 4            # tentativas por chamada antes de registrar o erro
TIMEOUT_S = 20

sessao = requests.Session()
registro = {"chamadas": 0, "chamadas_por_endpoint": {}, "novas_tentativas": 0, "erros": []}


def chamar_api(caminho, endpoint, **params):
    """GET em URL_API + caminho. Devolve o JSON da resposta ou None, registrando o erro.

    A Deezer devolve falhas como {"error": {...}} com HTTP 200: o código 4 (limite de chamadas)
    e falhas de rede ganham nova tentativa com espera crescente; os demais erros são registrados.
    """
    ultimo_erro = None
    for tentativa in range(1, TENTATIVAS + 1):
        time.sleep(PAUSA_S)
        registro["chamadas"] += 1
        registro["chamadas_por_endpoint"][endpoint] = registro["chamadas_por_endpoint"].get(endpoint, 0) + 1
        try:
            resposta = sessao.get(URL_API + caminho, params=params, timeout=TIMEOUT_S)
            resposta.raise_for_status()
            corpo = resposta.json()
        except (requests.RequestException, ValueError) as erro:
            ultimo_erro = {"tipo": type(erro).__name__, "mensagem": str(erro)[:200]}
        else:
            if not (isinstance(corpo, dict) and "error" in corpo):
                return corpo
```

```
            ultimo_erro = {"tipo": corpo["error"].get("type"), "codigo": corpo["error"].get("code"),
                           "mensagem": corpo["error"].get("message")}
            if ultimo_erro["codigo"] != 4:   # objeto inexistente ou parâmetro inválido: repetir não resolve
                break
        if tentativa < TENTATIVAS:
            registro["novas_tentativas"] += 1
            time.sleep(2 ** tentativa)
    registro["erros"].append({"caminho": caminho, "parametros": params, **ultimo_erro})
    return None


print(f"API: {URL_API} · limit={LIMITE_PAGINA} · teto por chart={TETO_POR_CHART} · pausa={PAUSA_S} s")
print(f"Coletar de novo na API: {BAIXAR_DA_API}")
```

```
# Coleta dos charts por gênero
if BAIXAR_DA_API:
    inicio_coleta = datetime.now(timezone.utc)
    infos = chamar_api("/infos", "/infos")   # país de onde a API enxerga a requisição

    resposta_generos = chamar_api("/genre", "/genre")
    if resposta_generos is None:
        raise RuntimeError(f"A API não devolveu a lista de gêneros: {registro['erros'][-1]}")
    generos = {g["id"]: g["name"] for g in resposta_generos["data"]}
    generos.setdefault(0, "All")   # chart geral

    charts = []
    for id_genero, nome_genero in generos.items():
        paginas = []
        for index in range(0, TETO_POR_CHART, LIMITE_PAGINA):
            pagina = chamar_api(f"/chart/{id_genero}/tracks", "/chart/{id}/tracks",
                                index=index, limit=LIMITE_PAGINA)
            if pagina is None:
                break
            paginas.append({"index": index, "limit": LIMITE_PAGINA, "resposta": pagina})
            lidos = len(pagina.get("data", []))
            if lidos < LIMITE_PAGINA or index + lidos >= pagina.get("total", index + lidos):
                break   # última página do chart
        charts.append({"id_genero": id_genero, "nome_genero": nome_genero, "paginas": paginas})
        itens = sum(len(p["resposta"].get("data", [])) for p in paginas)
        print(f"Chart {id_genero:>4} ({nome_genero}): {itens} itens em {len(paginas)} página(s)")
    print(f"{len(charts)} charts coletados em {registro['chamadas']} chamadas")
```

```
# Coleta dos detalhes de faixas, álbuns e artistas
if BAIXAR_DA_API:
    itens_chart = [item for c in charts for p in c["paginas"] for item in p["resposta"].get("data", [])]
    ids_faixas = sorted({item["id"] for item in itens_chart})
    ids_albuns = sorted({item["album"]["id"] for item in itens_chart if item.get("album")})
    ids_artistas = sorted({item["artist"]["id"] for item in itens_chart if item.get("artist")})
    print(f"{len(itens_chart)} itens nos charts · {len(ids_faixas)} faixas únicas · {len(ids_albuns)} álbuns únicos"
          f" · {len(ids_artistas)} artistas únicos")

    faixas = {}
    for i, id_faixa in enumerate(ids_faixas, 1):
        faixas[str(id_faixa)] = chamar_api(f"/track/{id_faixa}", "/track/{id}")
        if i % 250 == 0:
            print(f"  faixas: {i}/{len(ids_faixas)}")
    albuns = {}
    for i, id_album in enumerate(ids_albuns, 1):
        albuns[str(id_album)] = chamar_api(f"/album/{id_album}", "/album/{id}")
        if i % 250 == 0:
            print(f"  álbuns: {i}/{len(ids_albuns)}")
    artistas = {}
    for i, id_artista in enumerate(ids_artistas, 1):
        artistas[str(id_artista)] = chamar_api(f"/artist/{id_artista}", "/artist/{id}")
        if i % 250 == 0:
            print(f"  artistas: {i}/{len(ids_artistas)}")
    fim_coleta = datetime.now(timezone.utc)
    print(f"Coleta terminada: {registro['chamadas']} chamadas, {len(registro['erros'])} erro(s)")
```

```
# Gravação do retrato bruto
if BAIXAR_DA_API:
    pasta_retrato = PASTA_BRUTOS / inicio_coleta.strftime("%Y-%m-%d")
    pasta_retrato.mkdir(parents=True, exist_ok=True)
    conteudo = {"infos.json": infos, "generos.json": resposta_generos, "charts.json": charts,
                "faixas.json": faixas, "albuns.json": albuns, "artistas.json": artistas}
    for nome, dados in conteudo.items():
        # Respostas cruas da API, sem nenhuma alteração; JSON compacto para o retrato caber no repositório
        (pasta_retrato / nome).write_text(json.dumps(dados, ensure_ascii=False, separators=(",", ":")),
                                          encoding="utf-8")
    metadados = {
```

```
        "inicio_utc": inicio_coleta.isoformat(timespec="seconds"),
        "fim_utc": fim_coleta.isoformat(timespec="seconds"),
        "url_api": URL_API,
        "pais_aparente": (infos or {}).get("country_iso"),
        "endpoints": ["/infos", "/genre", "/chart/{id}/tracks", "/track/{id}", "/album/{id}", "/artist/{id}"],
        "parametros": {"limit": LIMITE_PAGINA, "teto_por_chart": TETO_POR_CHART, "pausa_s": PAUSA_S,
                       "tentativas": TENTATIVAS, "timeout_s": TIMEOUT_S},
        "chamadas": registro["chamadas"],
        "chamadas_por_endpoint": registro["chamadas_por_endpoint"],
        "novas_tentativas": registro["novas_tentativas"],
        "quantidade_erros": len(registro["erros"]),
        "erros": registro["erros"],
        "charts": len(charts),
        "itens_chart": len(itens_chart),
        "faixas_unicas": len(ids_faixas),
        "albuns_unicos": len(ids_albuns),
        "artistas_unicos": len(ids_artistas),
        "arquivos": sorted(conteudo) + ["metadados.json"],
        "ambiente": {"python": platform.python_version(), "pandas": pd.__version__,
                     "requests": requests.__version__, "colab": NO_COLAB},
    }
    (pasta_retrato / "metadados.json").write_text(json.dumps(metadados, ensure_ascii=False, indent=2),
                                                  encoding="utf-8")
    tamanho_mb = sum(a.stat().st_size for a in pasta_retrato.iterdir()) / 1e6
    print(f"Retrato gravado em {pasta_retrato.relative_to(RAIZ)} ({tamanho_mb:.1f} MB)")
```

```
# Leitura do retrato bruto mais recente
retratos = sorted(p for p in PASTA_BRUTOS.glob("????-??-??") if (p / "metadados.json").exists())
if not retratos:
    raise FileNotFoundError(
        f"Nenhum retrato bruto em {PASTA_BRUTOS}. Atualize o clone do repositório ou rode a seção 00b "
        "com BAIXAR_DA_API = True para coletar na API da Deezer."
    )
PASTA_RETRATO = retratos[-1]


def ler_json(nome):
    return json.loads((PASTA_RETRATO / nome).read_text(encoding="utf-8"))


metadados = ler_json("metadados.json")
charts = ler_json("charts.json")
faixas = ler_json("faixas.json")
albuns = ler_json("albuns.json")
artistas = ler_json("artistas.json")
print(f"Retrato lido: {PASTA_RETRATO.relative_to(RAIZ)} ({len(retratos)} retrato(s) disponível(is))")
for arquivo in sorted(PASTA_RETRATO.iterdir()):
    print(f"  {arquivo.name:<15} {arquivo.stat().st_size / 1e6:6.2f} MB")
```

```
# Metadados do retrato lido
resumo_metadados = pd.Series({
    "Início da coleta (UTC)": metadados["inicio_utc"],
    "Fim da coleta (UTC)": metadados["fim_utc"],
    "País aparente da requisição": metadados["pais_aparente"],
    "Endpoints": ", ".join(metadados["endpoints"]),
    "Parâmetros": ", ".join(f"{k}={v}" for k, v in metadados["parametros"].items()),
    "Chamadas à API": metadados["chamadas"],
    "Novas tentativas": metadados["novas_tentativas"],
    "Erros registrados": metadados["quantidade_erros"],
    "Charts coletados": metadados["charts"],
    "Itens nos charts": metadados["itens_chart"],
    "Faixas únicas": metadados["faixas_unicas"],
    "Álbuns únicos": metadados["albuns_unicos"],
    "Artistas únicos": metadados["artistas_unicos"],
    "Ambiente da coleta": ", ".join(f"{k} {v}" for k, v in metadados["ambiente"].items()),
}, name="valor")
display(resumo_metadados.to_frame())
display(pd.Series(metadados["chamadas_por_endpoint"], name="chamadas").to_frame())
if metadados["erros"]:
    display(pd.DataFrame(metadados["erros"]))
```

```
# Montagem do df_bruto
linhas = []
for chart in charts:
    ordem = 0
    for pagina in chart["paginas"]:
        for item in pagina["resposta"].get("data", []):
            ordem += 1
            linhas.append({"chart_id_genero": chart["id_genero"], "chart_nome_genero": chart["nome_genero"],
                           "chart_ordem": ordem, **item})
```

```
itens = pd.json_normalize(linhas, sep="_")


def campos_simples(objeto, prefixo, ignorar=()):
    """Campos de primeiro nível de uma resposta (objetos aninhados ficam de fora; listas ficam como vieram)."""
    return {f"{prefixo}{k}": v for k, v in objeto.items() if not isinstance(v, dict) and k not in ignorar}


detalhe_faixas = pd.DataFrame([campos_simples(f, "detalhe_faixa_") for f in faixas.values() if f])
detalhe_albuns = pd.DataFrame([
    {**campos_simples(a, "detalhe_album_", ignorar=("tracks",)),
     "detalhe_album_genres": (a.get("genres") or {}).get("data")}
    for a in albuns.values() if a
])
detalhe_artistas = pd.DataFrame([campos_simples(a, "detalhe_artista_") for a in artistas.values() if a])

df_bruto = (
    itens
    .merge(detalhe_faixas, how="left", left_on="id", right_on="detalhe_faixa_id")
    .merge(detalhe_albuns, how="left", left_on="album_id", right_on="detalhe_album_id")
    .merge(detalhe_artistas, how="left", left_on="artist_id", right_on="detalhe_artista_id")
)
print(f"df_bruto: {df_bruto.shape[0]} linhas × {df_bruto.shape[1]} colunas")
```

```
# Verificação do carregamento
origem_colunas = pd.Series(
    ["chart" if c.startswith("chart_") else "detalhe da faixa" if c.startswith("detalhe_faixa_")
     else "detalhe do álbum" if c.startswith("detalhe_album_")
     else "detalhe do artista" if c.startswith("detalhe_artista_") else "item do chart" for c in df_bruto.columns]
).value_counts().rename("colunas")
print(f"Linhas: {len(df_bruto)} (itens nos charts segundo os metadados: {metadados['itens_chart']})")
print(f"Faixas únicas: {df_bruto['id'].nunique()} · álbuns únicos: {df_bruto['album_id'].nunique()} · "
      f"artistas únicos: {df_bruto['artist_id'].nunique()}")
print(f"Itens sem detalhe da faixa: {df_bruto['detalhe_faixa_id'].isna().sum()} · "
      f"sem detalhe do álbum: {df_bruto['detalhe_album_id'].isna().sum()} · "
      f"sem detalhe do artista: {df_bruto['detalhe_artista_id'].isna().sum()}")
display(origem_colunas.to_frame())
display(df_bruto.groupby(["chart_id_genero", "chart_nome_genero"]).size().rename("itens").to_frame())
display(df_bruto.head())
```

3. Exploração inicial dos dados

Investigue a quantidade de registros e colunas, os tipos de dados, os valores ausentes e outras características relevantes da base.

Uma linha representa uma ocorrência de faixa em um chart no retrato da coleta. A chave candidata é o par `chart_id_genero` e `id`, dentro desse retrato. `id` identifica a faixa, mas não é único entre charts. As verificações abaixo contam ocorrências, não faixas distintas, salvo indicação explícita.

```
# Visão geral e tipos observados
import json
import numpy as np
import pandas as pd
from IPython.display import display

numero_linhas = len(df_bruto)
if numero_linhas == 0:
    raise ValueError("O diagnóstico exige um df_bruto não vazio; execute a seção 2.")
print(f"Retrato: {numero_linhas} linhas × {df_bruto.shape[1]} colunas; "
      f"{df_bruto['id'].nunique()} faixas; {df_bruto['chart_id_genero'].nunique()} charts.")
print(f"Memória, incluindo objetos: {df_bruto.memory_usage(deep=True).sum():,} bytes.")
inventario = df_bruto.dtypes.astype(str).rename("tipo_observado").to_frame()
display(inventario.reset_index(names="atributo"))
```

```
# Ausências nulas e textos vazios
# isna() encontra nulos. Para textos, também contamos vazio e espaços como ausência.
mascara_ausencias = df_bruto.isna()
textos_vazios = pd.DataFrame(False, index=df_bruto.index, columns=df_bruto.columns)

for coluna in df_bruto.columns:
    if pd.api.types.is_string_dtype(df_bruto[coluna].dropna()):
        textos_vazios[coluna] = df_bruto[coluna].str.strip().eq("").fillna(False)
        mascara_ausencias[coluna] = mascara_ausencias[coluna] | textos_vazios[coluna]

ausencias = pd.DataFrame({
    "nulos": df_bruto.isna().sum(),
    "textos_vazios": textos_vazios.sum(),
    "total_ausentes": mascara_ausencias.sum(),
})
```

```
ausencias["percentual"] = 100 * ausencias["total_ausentes"] / numero_linhas
display(ausencias.reset_index(names="atributo"))
```

```
# Duplicidade e limite da chave
# Algumas colunas guardam listas. Só para comparar linhas, transformamos listas em texto.
# df_bruto continua intacto: a comparação usa outra tabela.
def texto_para_comparacao(valor):
    if isinstance(valor, (list, dict)):
        return json.dumps(valor, ensure_ascii=False, sort_keys=True)
    return valor

tabela_comparacao = df_bruto.copy()
for coluna in tabela_comparacao.columns:
    tabela_comparacao[coluna] = tabela_comparacao[coluna].map(texto_para_comparacao)

duplicadas_inteiras = tabela_comparacao.duplicated()
duplicadas_id = df_bruto.duplicated("id")
duplicadas_chart = df_bruto.duplicated(["chart_id_genero", "id"])
print(f"Ocorrências excedentes por id: {duplicadas_id.sum()}/{numero_linhas}; "
      f"por par chart-faixa: {duplicadas_chart.sum()}/{numero_linhas}; "
      f"linhas inteiras iguais: {duplicadas_inteiras.sum()}/{numero_linhas}.")
```

```
# Estatísticas descritivas e domínios numéricos
colunas_numericas = ["duration", "rank", "position", "detalhe_faixa_bpm",
                     "detalhe_faixa_gain", "detalhe_artista_nb_fan"]
numericos = df_bruto[colunas_numericas].apply(pd.to_numeric, errors="coerce")
display(numericos.describe().T)
# São regras de integridade, não indicadores analíticos das perguntas P1 a P4.
colunas_inteiras = ["id", "chart_id_genero", "chart_ordem", "duration", "rank", "position",
                    "artist_id", "album_id", "detalhe_faixa_id", "detalhe_album_id",
                    "detalhe_artista_id", "detalhe_artista_nb_fan"]
tipos_invalidos = pd.Series(False, index=df_bruto.index)
for coluna in colunas_inteiras:
    valores = pd.to_numeric(df_bruto[coluna], errors="coerce")
    tipos_invalidos |= valores.isna() | ~np.isfinite(valores) | valores.mod(1).ne(0)
# Strings conversíveis também são divergências do tipo esperado, mesmo sem perda na conversão.
for coluna in colunas_inteiras + ["detalhe_faixa_bpm", "detalhe_faixa_gain"]:
    tipos_invalidos |= ~df_bruto[coluna].map(
        lambda valor: isinstance(valor, (int, float, np.integer, np.floating))
        and not isinstance(valor, (bool, np.bool_))
    )
tipos_invalidos |= ~df_bruto["explicit_lyrics"].map(lambda valor: isinstance(valor, (bool, np.bool_)))
# Cada comparação devolve True nas linhas com problema.
invalidos_numericos = numericos.isna().any(axis=1) | ~np.isfinite(numericos).all(axis=1)
invalidos_numericos = invalidos_numericos | (numericos["duration"] <= 0)
invalidos_numericos = invalidos_numericos | (numericos["position"] <= 0)
invalidos_numericos = invalidos_numericos | (numericos["rank"] < 0)
invalidos_numericos = invalidos_numericos | (numericos["detalhe_artista_nb_fan"] < 0)
invalidos_numericos = invalidos_numericos | (numericos["detalhe_faixa_bpm"] < 0)
for coluna in ["id", "artist_id", "album_id"]:
    valores = pd.to_numeric(df_bruto[coluna], errors="coerce")
    invalidos_numericos = invalidos_numericos | (valores <= 0)
bpm_zero = numericos["detalhe_faixa_bpm"] == 0
gain_zero = numericos["detalhe_faixa_gain"] == 0
# gain é um ajuste em dB: negativo é permitido; zero é sinalizado para avaliação, não prova de erro.
```

```
# Datas, formatos e categorias
colunas_datas = ["detalhe_faixa_release_date", "detalhe_album_release_date"]
# A data de comparação vem dos metadados da coleta, para repetir o mesmo resultado depois.
data_coleta = pd.Timestamp(metadados["inicio_utc"]).date()
datas_invalidas = pd.Series(False, index=df_bruto.index)
for coluna in colunas_datas:
    texto = df_bruto[coluna].astype("string")
    formato_correto = texto.str.fullmatch(r"\d{4}-\d{2}-\d{2}").fillna(False)
    datas = pd.to_datetime(texto, format="%Y-%m-%d", errors="coerce")
    problema = ~formato_correto | datas.isna()
    problema = problema | (datas < pd.Timestamp("1900-01-01"))
    problema = problema | (datas > pd.Timestamp(data_coleta))
    datas_invalidas = datas_invalidas | problema

espacos = pd.Series(False, index=df_bruto.index)
for coluna in ["title", "artist_name", "chart_nome_genero"]:
    texto = df_bruto[coluna].astype("string")
    espacos = espacos | texto.ne(texto.str.strip()).fillna(False)
nomes_por_artista = df_bruto.groupby("artist_id")["artist_name"].nunique()
variacoes_artista = df_bruto["artist_id"].isin(nomes_por_artista[nomes_por_artista.gt(1)].index)
nomes_por_genero = df_bruto.groupby("chart_id_genero")["chart_nome_genero"].nunique()
categorias_inconsistentes = df_bruto["chart_id_genero"].isin(nomes_por_genero[nomes_por_genero.gt(1)].index)
nomes_normalizados = df_bruto["chart_nome_genero"].astype("string").str.strip().str.casefold()
ids_por_nome = df_bruto.groupby(nomes_normalizados)["chart_id_genero"].nunique()
```

```
categorias_inconsistentes |= nomes_normalizados.isin(ids_por_nome[ids_por_nome.gt(1)].index)
chart_geral = df_bruto["chart_id_genero"].eq(0)
genero_album_generico = pd.to_numeric(df_bruto["detalhe_album_genre_id"], errors="coerce").le(0)
```

```
# Possíveis outliers e integridade do enriquecimento
# Limites IQR são sinais para inspeção; extremos não são removidos.
limites_outliers = []
for coluna in ["duration", "rank"]:
    primeiro_quartil, terceiro_quartil = numericos[coluna].quantile([0.25, 0.75])
    amplitude = terceiro_quartil - primeiro_quartil
    inferior, superior = primeiro_quartil - 1.5 * amplitude, terceiro_quartil + 1.5 * amplitude
    mascara = numericos[coluna].lt(inferior) | numericos[coluna].gt(superior)
    limites_outliers.append({"atributo": coluna, "inferior": inferior, "superior": superior,
                             "ocorrencias": int(mascara.sum()), "denominador": numero_linhas})
    if coluna == "duration":
        outliers_duracao = mascara
    else:
        outliers_rank = mascara
display(pd.DataFrame(limites_outliers))
display(df_bruto.loc[outliers_duracao, ["id", "title", "duration", "chart_nome_genero"]])
integridade = pd.Series(False, index=df_bruto.index)
for origem, detalhe in [("id", "detalhe_faixa_id"), ("album_id", "detalhe_album_id"),
                        ("artist_id", "detalhe_artista_id")]:
    integridade |= df_bruto[origem].isna() | df_bruto[detalhe].isna() | df_bruto[origem].ne(df_bruto[detalhe])
divergencia_duracao = df_bruto["duration"].ne(df_bruto["detalhe_faixa_duration"])
divergencia_rank = df_bruto["rank"].ne(df_bruto["detalhe_faixa_rank"])
```

```
# Diagnóstico de qualidade Q1 a Q21
campos_essenciais = ["id", "chart_id_genero", "chart_nome_genero", "title", "duration", "rank",
                     "position", "artist_id", "artist_name", "album_id", "detalhe_faixa_release_date",
                     "detalhe_artista_nb_fan"]
verificacoes = [
    ("Q1", "Ausência em pelo menos um campo essencial às P1–P4 ou RA2", mascara_ausencias[campos_essenciais].any(axis=1), '
    ("Q2", "Ausência de preview (nulo ou texto vazio)", mascara_ausencias["preview"], "Ausências nulas e textos vazios"),
    ("Q3", "Ausência de title_version (nulo ou texto vazio)", mascara_ausencias["title_version"], "Ausências nulas e textos
    ("Q4", "Ocorrências excedentes por id, após a primeira", duplicadas_id, "Duplicidade e limite da chave"),
    ("Q5", "Ocorrências excedentes pelo par chart_id_genero e id", duplicadas_chart, "Duplicidade e limite da chave"),
    ("Q6", "Linhas inteiras repetidas, após a primeira", duplicadas_inteiras, "Duplicidade e limite da chave"),
    ("Q7", "Tipo divergente de numérico inteiro/real ou booleano esperado", tipos_invalidos, "Estatísticas descritivas e dc
    ("Q8", "Numéricos não finitos/ausentes ou fora dos domínios descritos", invalidos_numericos, "Estatísticas descritivas
    ("Q9", "bpm igual a zero (sinal de dado não informado)", bpm_zero, "Estatísticas descritivas e domínios numéricos"),
    ("Q10", "gain igual a zero (a avaliar, não erro comprovado)", gain_zero, "Estatísticas descritivas e domínios numéricos
    ("Q11", "Data fora de AAAA-MM-DD, inválida ou fora de 1900 até a coleta", datas_invalidas, "Datas, formatos e categoria
    ("Q12", "Espaços nas pontas de título, artista ou gênero", espacos, "Datas, formatos e categorias"),
    ("Q13", "Mais de um nome para um id de artista", variacoes_artista, "Datas, formatos e categorias"),
    ("Q14", "Gênero com mais de um nome ou nome normalizado com mais de um id", categorias_inconsistentes, "Datas, formatos
    ("Q15", "Chart geral id 0: não representa um gênero específico", chart_geral, "Datas, formatos e categorias"),
    ("Q16", "Gênero do álbum genérico (id menor ou igual a zero)", genero_album_generico, "Datas, formatos e categorias"),
    ("Q17", "Duração fora de Q1 − 1,5 IQR a Q3 + 1,5 IQR", outliers_duracao, "Possíveis outliers e integridade do enriquec
    ("Q18", "Rank fora de Q1 − 1,5 IQR a Q3 + 1,5 IQR", outliers_rank, "Possíveis outliers e integridade do enriquecimento"
    ("Q19", "Id ausente ou divergente na junção de faixa/álbum/artista", integridade, "Possíveis outliers e integridade do
    ("Q20", "Duração do chart diferente da duração em /track", divergencia_duracao, "Possíveis outliers e integridade do en
    ("Q21", "Rank do chart diferente do rank em /track", divergencia_rank, "Possíveis outliers e integridade do enriquecime
]
# Somar uma coluna de True/False conta quantas linhas apresentaram o problema.
resultados = []
for identificador, regra, mascara, celula in verificacoes:
    quantidade = int(mascara.sum())
    resultados.append({
        "ID": identificador,
        "regra": regra,
        "quantidade": quantidade,
        "denominador": numero_linhas,
        "percentual": 100 * quantidade / numero_linhas,
        "celula": celula,
    })

diagnostico = pd.DataFrame(resultados)
display(diagnostico)
print("As proporções usam ocorrências nos charts e podem se sobrepor; não devem ser somadas.")
```

Resposta do grupo:

No retrato de 16/10/2026, são 2.800 ocorrências, 102 colunas e 2.581 faixas. Há 219/2.800 repetições excedentes por `id` (7,82%), mas nenhuma pela chave chart-faixa. BPM é zero em 1.890/2.800 ocorrências (67,50%); `title_version` está ausente em 2.533/2.800 (90,46%). Uma ocorrência tem lançamento posterior à coleta, e 143/2.800 (5,11%) têm duração fora dos limites IQR. Evidências: células “Visão geral e tipos observados” e “Diagnóstico de qualidade Q1 a Q21”. A repetição por `id` exige preservar o chart de origem na preparação, pois ele define os grupos das perguntas P1 a P4. O chart geral deve ser identificado separadamente dos gêneros. BPM

zero e versões sem texto precisam de interpretação; valores extremos de duração são candidatos à inspeção, sem exclusão automática. O diagnóstico completo e as prioridades estão no documento “Compreensão e qualidade”, com as mesmas contagens do quadro Q1 a Q21.

4. Limpeza e preparação dos dados

Realize a limpeza necessária ao projeto. Verifique valores ausentes, duplicados e inconsistências e trate os problemas encontrados, quando necessário.

As decisões D1–D8 respondem às verificações Q1–Q21. A preparação preserva a participação de cada faixa nos charts, corrige espaços e sinaliza situações que exigem cuidado, sem excluir automaticamente dados por critérios estatísticos.

```
# Cópia de trabalho e referência do bruto
import numpy as np
import pandas as pd
from IPython.display import display

referencia_bruta_preparacao = df_bruto.copy(deep=True)
df_preparado = df_bruto.copy(deep=True)
total_antes_preparacao = len(df_bruto)
assert total_antes_preparacao > 0, "Execute a ingestão antes de preparar os dados."
print(f"Entrada: {total_antes_preparacao} ocorrências × {df_bruto.shape[1]} atributos.")
```

```
# Preservação do vínculo entre faixa e chart
# D1 (Q4–Q6): a faixa pode se repetir entre charts, mas a chave composta é única.
chave_bruta = ["chart_id_genero", "id"]
repeticoes_id_antes = int(df_bruto.duplicated("id").sum())
duplicatas_chave_antes = int(df_bruto.duplicated(chave_bruta).sum())
assert df_bruto[chave_bruta].notna().all().all(), "Chave de origem incompleta."
assert duplicatas_chave_antes == 0, "Revise o diagnóstico: há pares chart-faixa repetidos."
pd.testing.assert_frame_equal(df_preparado[chave_bruta], df_bruto[chave_bruta])
print(f"D1: {repeticoes_id_antes}/{total_antes_preparacao} excedentes por id mantidos; "
      f"{duplicatas_chave_antes}/{total_antes_preparacao} duplicatas chart-faixa.")
```

```
# Padronização dos campos textuais
# D2 (Q12): remover somente espaços nas pontas, preservando acentos e a escrita.
colunas_textuais_preparacao = ["title", "artist_name", "chart_nome_genero"]
espacos_antes_preparacao = pd.Series(False, index=df_bruto.index)
alteracoes_textuais_preparacao = []
for coluna in colunas_textuais_preparacao:
    antes = df_preparado[coluna].astype("string")
    depois = antes.str.strip()
    alteradas = antes.ne(depois).fillna(False)
    espacos_antes_preparacao |= alteradas
    for indice in df_preparado.index[alteradas]:
        alteracoes_textuais_preparacao.append({
            "id_faixa": int(df_preparado.at[indice, "id"]),
            "campo": coluna, "antes": repr(antes.at[indice]),
            "depois": repr(depois.at[indice]),
        })
    df_preparado[coluna] = depois
    assert depois.eq(depois.str.strip()).fillna(True).all()
print(f"D2: {int(espacos_antes_preparacao.sum())}/{total_antes_preparacao} "
      "ocorrências corrigidas; nenhuma linha excluída.")
display(pd.DataFrame(alteracoes_textuais_preparacao).head(10))
```

```
# Conversão e validação temporal
# D3 (Q11): conservar o lançamento e identificar datas posteriores à coleta.
texto_lancamento_preparacao = df_preparado["detalhe_faixa_release_date"].astype("string")
assert texto_lancamento_preparacao.str.fullmatch(r"\d{4}-\d{2}-\d{2}").fillna(False).all()
datas_lancamento_preparacao = pd.to_datetime(
    texto_lancamento_preparacao, format="%Y-%m-%d", errors="raise"
)
assert datas_lancamento_preparacao.notna().all()
assert datas_lancamento_preparacao.ge(pd.Timestamp("1900-01-01")).all()
assert datas_lancamento_preparacao.dt.strftime("%Y-%m-%d").eq(
    texto_lancamento_preparacao
).all()
data_retrato_preparacao = pd.Timestamp(metadados["inicio_utc"])
if data_retrato_preparacao.tzinfo is not None:
    data_retrato_preparacao = data_retrato_preparacao.tz_convert("UTC").tz_localize(None)
data_retrato_preparacao = data_retrato_preparacao.normalize()
df_preparado["detalhe_faixa_release_date"] = datas_lancamento_preparacao
df_preparado["data_coleta"] = data_retrato_preparacao
df_preparado["lancamento_posterior_coleta"] = (
    datas_lancamento_preparacao > data_retrato_preparacao
```

```
)
print(f"D3: {int(df_preparado['lancamento_posterior_coleta'].sum())}/"
      f"{total_antes_preparacao} lançamentos posteriores à coleta, preservados.")
display(df_preparado.loc[
    df_preparado["lancamento_posterior_coleta"],
    ["id", "title", "detalhe_faixa_release_date", "data_coleta"],
].head(10))
```

```
# Identificação do chart de origem
# D4 (Q14–Q16): o gênero do álbum não substitui o chart de origem.
df_preparado["tipo_chart"] = df_preparado["chart_id_genero"].map(
    lambda identificador: "geral" if identificador == 0 else "genero"
)
charts_gerais_preparacao = df_preparado["chart_id_genero"].eq(0)
assert df_preparado["tipo_chart"].eq("geral").equals(charts_gerais_preparacao)
assert df_preparado.groupby("chart_id_genero")["chart_nome_genero"].nunique().eq(1).all()
print(f"D4: {int(charts_gerais_preparacao.sum())}/{total_antes_preparacao} "
      "ocorrências identificadas como chart geral, sem exclusão.")
```

```
# Sinalização dos extremos de duração
# D7 (Q17–Q18): reproduzir a regra da seção 3, sem alterar duração ou rank.
quartis_duracao_preparacao = df_bruto["duration"].quantile([0.25, 0.75])
amplitude_duracao_preparacao = quartis_duracao_preparacao.iloc[1] - (
    quartis_duracao_preparacao.iloc[0]
)
limite_inferior_duracao = quartis_duracao_preparacao.iloc[0] - (
    1.5 * amplitude_duracao_preparacao
)
limite_superior_duracao = quartis_duracao_preparacao.iloc[1] + (
    1.5 * amplitude_duracao_preparacao
)
df_preparado["duracao_fora_limites_iqr"] = (
    df_preparado["duration"].lt(limite_inferior_duracao)
    | df_preparado["duration"].gt(limite_superior_duracao)
)
pd.testing.assert_series_equal(df_preparado["duration"], df_bruto["duration"])
pd.testing.assert_series_equal(df_preparado["rank"], df_bruto["rank"])
print(f"D7: limites {limite_inferior_duracao:g} a {limite_superior_duracao:g} s; "
      f"{int(df_preparado['duracao_fora_limites_iqr'].sum())}/"
      f"{total_antes_preparacao} ocorrências sinalizadas e mantidas.")
```

```
# Registro das decisões de preparação
decisoes_preparacao = pd.DataFrame([
    ("D1", "Q4–Q6", "Preservar o vínculo chart-faixa", "Nenhuma linha excluída"),
    ("D2", "Q12", "Remover espaços nas pontas", "Preservar a escrita dos nomes"),
    ("D3", "Q11", "Converter e sinalizar datas", "Não imputar lançamento"),
    ("D4", "Q14–Q16", "Identificar o tipo de chart", "Preservar a origem"),
    ("D5", "Q2, Q3, Q9, Q10", "Selecionar atributos na seção 5", "Não excluir faixas"),
    ("D6", "Q7, Q8", "Padronizar nomes e tipos na seção 5", "Não alterar números válidos"),
    ("D7", "Q17, Q18", "Sinalizar extremos", "Não cortar ou substituir valores"),
    ("D8", "Q1, Q13, Q19–Q21", "Revalidar consistência na seção 5", "Manter dados aprovados"),
], columns=["decisao", "diagnostico", "acao", "justificativa"])
display(decisoes_preparacao)
pd.testing.assert_frame_equal(df_bruto, referencia_bruta_preparacao)
assert len(df_preparado) == total_antes_preparacao
```

Resposta do grupo:

Preservamos as 2.800 ocorrências e o vínculo faixa-chart (D1), com 219/2.800 excedentes por id e nenhuma duplicata pela chave composta. Corrigimos espaços nas pontas em 1/2.800 ocorrências (D2), convertemos o lançamento e sinalizamos 1/2.800 datas posteriores à coleta (D3). Identificamos separadamente as 100/2.800 ocorrências do chart geral (D4). As 143/2.800 durações fora dos limites IQR foram sinalizadas, sem exclusão nem substituição (D7).

Evidências: células “Preservação do vínculo entre faixa e chart”, “Padronização dos campos textuais”, “Conversão e validação temporal”, “Identificação do chart de origem” e “Sinalização dos extremos de duração”. A seção 5 conclui D5, D6 e D8 com seleção, padronização, validação e gravação. O bruto permanece intacto.

5. Transformação dos dados

Realize as transformações necessárias à análise, como criação de colunas, filtros, agrupamentos, conversões ou junções, quando aplicáveis.

Selecionamos os atributos, padronizamos nomes e tipos e gravamos o conjunto preparado. A unidade continua sendo uma ocorrência de faixa por chart e data de coleta. As perguntas analíticas serão respondidas posteriormente.

```
# Seleção e organização dos atributos
# D5: doze atributos do bruto e quatro campos de contexto ou qualidade.
nomes_colunas_preparadas = {
    "id": "id_faixa", "title": "titulo", "chart_id_genero": "id_chart",
    "chart_nome_genero": "genero", "tipo_chart": "tipo_chart",
    "position": "posicao_chart", "artist_id": "id_artista", "artist_name": "artista",
    "album_id": "id_album", "duration": "duracao_s", "rank": "rank",
    "detalhe_artista_nb_fan": "numero_fas",
    "detalhe_faixa_release_date": "data_lancamento", "data_coleta": "data_coleta",
    "lancamento_posterior_coleta": "lancamento_posterior_coleta",
    "duracao_fora_limites_iqr": "duracao_fora_limites_iqr",
}
df_preparado = df_preparado.loc[:, list(nomes_colunas_preparadas)].rename(
    columns=nomes_colunas_preparadas
)
assert df_preparado.columns.tolist() == list(nomes_colunas_preparadas.values())
assert len(df_preparado) == total_antes_preparacao
print(f"D5: {df_bruto.shape[1]} atributos de origem → "
      f"{df_preparado.shape[1]} atributos finais; {len(df_preparado)} linhas preservadas.")
```

```
# Padronização dos nomes e tipos
# D6: rank mantém o nome já esperado pelo gerador de eventos da seção 9.
colunas_inteiras_preparadas = [
    "id_faixa", "id_chart", "posicao_chart", "id_artista", "id_album",
    "duracao_s", "rank", "numero_fas",
]
colunas_textuais_preparadas = ["titulo", "genero", "tipo_chart", "artista"]
colunas_datas_preparadas = ["data_lancamento", "data_coleta"]
colunas_booleanas_preparadas = [
    "lancamento_posterior_coleta", "duracao_fora_limites_iqr",
]
for coluna in colunas_inteiras_preparadas:
    valores = pd.to_numeric(df_preparado[coluna], errors="raise")
    assert valores.notna().all() and np.isfinite(valores).all(), coluna
    assert valores.mod(1).eq(0).all(), f"Inteiro fracionário em {coluna}."
    df_preparado[coluna] = valores.astype("int64")
for coluna in colunas_textuais_preparadas:
    df_preparado[coluna] = df_preparado[coluna].astype("string")
for coluna in colunas_datas_preparadas:
    df_preparado[coluna] = pd.to_datetime(
        df_preparado[coluna], errors="raise"
    ).astype("datetime64[ns]")
for coluna in colunas_booleanas_preparadas:
    assert pd.api.types.is_bool_dtype(df_preparado[coluna]), coluna
display(df_preparado.dtypes.astype(str).rename("tipo").to_frame())
```

```
# Verificações finais do conjunto preparado
# D8: falhas são expostas; não há preenchimento automático nem remoção de linhas.
assert df_preparado.notna().all().all(), "O conjunto selecionado tem valores ausentes."
chave_preparada = ["data_coleta", "id_chart", "id_faixa"]
assert not df_preparado.duplicated(chave_preparada).any()
for coluna in ["id_faixa", "id_artista", "id_album", "duracao_s", "posicao_chart"]:
    assert df_preparado[coluna].gt(0).all(), coluna
for coluna in ["id_chart", "rank", "numero_fas"]:
    assert df_preparado[coluna].ge(0).all(), coluna
for coluna in colunas_textuais_preparadas:
    assert df_preparado[coluna].str.strip().ne("").all(), coluna
    assert df_preparado[coluna].eq(df_preparado[coluna].str.strip()).all(), coluna
assert df_preparado["tipo_chart"].isin(["geral", "genero"]).all()
assert df_preparado["tipo_chart"].eq("geral").eq(df_preparado["id_chart"].eq(0)).all()
assert df_preparado.groupby("id_chart")["genero"].nunique().eq(1).all()
assert df_preparado.groupby("id_artista")["artista"].nunique().eq(1).all()
assert df_preparado["lancamento_posterior_coleta"].equals(
    df_preparado["data_lancamento"].gt(df_preparado["data_coleta"])
)
assert df_preparado["duracao_fora_limites_iqr"].equals(
    df_preparado["duracao_s"].lt(limite_inferior_duracao)
    | df_preparado["duracao_s"].gt(limite_superior_duracao)
)
# A correspondência linha a linha garante que a seleção não perdeu a origem.
for origem, destino in nomes_colunas_preparadas.items():
    if destino in colunas_inteiras_preparadas:
        pd.testing.assert_series_equal(
            df_preparado[destino], df_bruto[origem].astype("int64"), check_names=False
        )
for origem, detalhe in [
    ("id", "detalhe_faixa_id"), ("album_id", "detalhe_album_id"),
    ("artist_id", "detalhe_artista_id"),
]:
    assert df_bruto[origem].eq(df_bruto[detalhe]).all(), detalhe
assert df_preparado["duracao_s"].eq(df_bruto["detalhe_faixa_duration"]).all()
```

```
assert df_preparado["rank"].eq(df_bruto["detalhe_faixa_rank"]).all()
assert df_preparado["data_lancamento"].dt.strftime("%Y-%m-%d").eq(
    df_bruto["detalhe_faixa_release_date"].astype("string")
).all()
assert df_preparado["data_coleta"].eq(data_retrato_preparacao).all()
pd.testing.assert_frame_equal(df_bruto, referencia_bruta_preparacao)
print(f"Validações aprovadas em {len(df_preparado)} ocorrências: "
      "tipos, domínios, ausências, chave, datas, sinalizações e integridade.")
```

```
# Comparação antes e depois da preparação
espacos_depois_preparacao = pd.Series(False, index=df_preparado.index)
for coluna in colunas_textuais_preparadas:
    espacos_depois_preparacao |= df_preparado[coluna].ne(
        df_preparado[coluna].str.strip()
    )
comparacao_preparacao = pd.DataFrame([
    ("Registros", total_antes_preparacao, len(df_preparado)),
    ("Atributos", df_bruto.shape[1], df_preparado.shape[1]),
    ("Ocorrências com espaços nas pontas", int(espacos_antes_preparacao.sum()),
     int(espacos_depois_preparacao.sum())),
    ("Duplicatas chart-faixa", duplicatas_chave_antes,
     int(df_preparado.duplicated(chave_preparada).sum())),
    ("Excedentes por id da faixa", repeticoes_id_antes,
     int(df_preparado.duplicated("id_faixa").sum())),
    ("Ocorrências do chart geral", int(charts_gerais_preparacao.sum()),
     int(df_preparado["tipo_chart"].eq("geral").sum())),
    ("Lançamentos posteriores à coleta",
     int(datas_lancamento_preparacao.gt(data_retrato_preparacao).sum()),
     int(df_preparado["lancamento_posterior_coleta"].sum())),
    ("Durações fora dos limites IQR",
     int((df_bruto["duration"].lt(limite_inferior_duracao)
          | df_bruto["duration"].gt(limite_superior_duracao)).sum()),
     int(df_preparado["duracao_fora_limites_iqr"].sum())),
], columns=["medida", "antes", "depois"])
display(comparacao_preparacao)
print(f"Denominador das medidas por ocorrência: {total_antes_preparacao} antes "
      f"e {len(df_preparado)} depois; as medidas podem se sobrepor.")
```

```
# Gravação e releitura do conjunto preparado
arquivo_preparado = PASTA_PREPARADOS / "faixas_preparadas.csv"
df_preparado.to_csv(arquivo_preparado, index=False, encoding="utf-8", date_format="%Y-%m-%d")
tipos_releitura_preparacao = {
    **{coluna: "int64" for coluna in colunas_inteiras_preparadas},
    **{coluna: "string" for coluna in colunas_textuais_preparadas},
    **{coluna: "bool" for coluna in colunas_booleanas_preparadas},
}
releitura_preparacao = pd.read_csv(
    arquivo_preparado, encoding="utf-8", dtype=tipos_releitura_preparacao,
    parse_dates=colunas_datas_preparadas,
)
pd.testing.assert_frame_equal(df_preparado, releitura_preparacao)
print(f"Arquivo: {arquivo_preparado.relative_to(RAIZ)} · UTF-8, separador vírgula.")
print(f"Releitura validada: {len(releitura_preparacao)} linhas × "
      f"{releitura_preparacao.shape[1]} colunas, sem perda de valores.")
display(df_preparado.head(5))
```

Resposta do grupo:

O conjunto preparado está em `dados/preparados/faixas_preparadas.csv`, com 2.800 ocorrências e 16 colunas: 12 atributos selecionados do bruto e quatro campos de contexto ou qualidade. Nenhuma linha foi excluída. Os espaços nas pontas passaram de 1/2.800 para 0/2.800 ocorrências. A chave data-chart-faixa permaneceu sem duplicatas; as 219/2.800 repetições por id foram preservadas. Evidências: células “Comparação antes e depois da preparação” e “Gravação e releitura do conjunto preparado”.

As 100/2.800 ocorrências do chart geral estão identificadas em `tipo_chart`. A data futura (1/2.800) e os extremos de duração (143/2.800) foram mantidos e sinalizados, sem confirmação de suas causas. O CSV não inclui BPM, ganho, prévia ou versão complementar; esses dados continuam no bruto. O gerador do RA2 precisa filtrar o chart geral ao usar o preparado, mantendo somente `tipo_chart == "genero"` para os eventos por gênero.

7. Considerações do RA1

Registre brevemente:

- O que o grupo descobriu sobre os dados.
- Quais dificuldades ou limitações encontrou.
- Como os dados preparados poderão ser utilizados nas próximas etapas.

Resposta do grupo:

O que descobrimos sobre os dados. O retrato de 16/10/2026 tem 2.800 ocorrências de 2.581 faixas em 28 charts, o geral e 27 gêneros. Nos campos que as perguntas usam, a qualidade de origem é boa: nenhuma ausência nos campos essenciais, nenhum tipo divergente e a mesma duração e o mesmo `rank` no chart e no detalhe da faixa nas 2.800 ocorrências (Q1, Q7, Q20 e Q21). Os problemas estão em campos fora das perguntas: `bpm` zerado em 1.890/2.800 ocorrências (67,50%) e `title_version` vazio em 2.533/2.800 (90,46%). Uma faixa pode estar em mais de um chart (219/2.800 ocorrências excedentes por `id`, 7,82%), por isso cada linha do conjunto preparado é uma faixa num chart. Evidência: seção 3, célula "Diagnóstico de qualidade Q1 a Q21".

Dificuldades e limitações. O chart muda todo dia e varia com o país: os números valem para um único retrato, coletado com país aparente US (seção 2). A API não informa reproduções, só o `rank`, um índice calculado pela Deezer. Uma data de lançamento posterior à coleta (1/2.800) e 143/2.800 durações fora dos limites IQR (5,11%) foram mantidas e sinalizadas, sem confirmar a causa. As 100/2.800 ocorrências do chart geral não representam um gênero. `bpm`, ganho, prévia e versão ficaram fora do conjunto preparado e continuam só no retrato bruto. Evidências: seção 4, células "Conversão e validação temporal", "Identificação do chart de origem" e "Sinalização dos extremos de duração".

Uso nas próximas etapas. O conjunto `dados/preparados/faixas_preparadas.csv` tem 2.800 linhas e 16 colunas, gravado e relido sem perda (seção 5, célula "Gravação e releitura do conjunto preparado"). No RA2, as 2.700 ocorrências dos charts de gênero alimentam os eventos de reprodução simulados (seção 9, célula "Faixas que alimentam os eventos"). No RA3, `duracao_s`, `data_lancamento`, `rank`, `numero_fas`, `id_artista`, `posicao_chart` e `genero` atendem às perguntas P1 a P4; `tipo_chart` separa o chart geral, e `lancamento_posterior_coleta` e `duracao_fora_limites_iqr` permitem decidir, na análise, como tratar a data futura e os extremos.

RA2 — Streaming Data Processing and Time Windows

Utilize os dados e o contexto do projeto para demonstrar o processamento em streaming e a aplicação de janelas temporais. Escolha um dos cenários:

- **Cenário A — Dados estáticos:** simule a chegada de novos registros utilizando dados artificiais coerentes com o contexto do projeto.
- **Cenário B — Dados em streaming:** utilize sua própria fonte de dados contínuos, sem necessidade de simulação.

8. Objetivo da análise em streaming

Responda brevemente:

- O que o grupo deseja monitorar ou analisar?
- Qual informação espera obter com o processamento em streaming?

Resposta do grupo:

No RA1, o grupo compara entre gêneros o perfil das faixas que estão no topo dos charts da Deezer. No RA2, o grupo quer **monitorar as reproduções dessas faixas por gênero ao longo do tempo**. Como a API da Deezer não oferece um fluxo de reproduções, usamos o cenário A: eventos de reprodução simulados, sorteados entre as faixas e os gêneros do conjunto preparado do RA1.

Com o processamento em streaming, esperamos saber, a cada janela de tempo, **quantas reproduções cada gênero acumula** e **qual gênero concentra mais reproduções naquele intervalo**, e ver esses números mudarem conforme novos lotes de eventos chegam. O tipo e a duração da janela estão na seção 10.

9. Fonte de dados e processamento

- Informe o cenário escolhido: dados reais em streaming ou simulação.
- Explique brevemente como os novos registros chegam.
- Implemente o processamento em tempo real.
- Demonstre o recebimento de novos registros.

Resposta do grupo:

Usamos o cenário A, de simulação, porque a API da Deezer não oferece um fluxo de reproduções em tempo real (ADR-0005). Cada registro novo é uma reprodução de faixa, com `id_evento`, `id_faixa`, `genero`, `artista`, `duracao_ouvida_s` e `instante`. A função `gerar_eventos` sorteia faixas e gêneros reais do conjunto do RA1, com chance proporcional ao `rank` da Deezer, semente 42 e primeiro instante em 21/10/2026 às 10:00, e espaça os eventos de 5 a 30 segundos.

Os registros chegam em lotes: `publicar_lote` grava cada lote como um arquivo JSON, com um evento por linha, e só então o move para `dados/streaming/entrada/`, a pasta que o Spark monitora com `spark.readStream` e esquema explícito. A consulta de recebimento conta os eventos que chegaram: antes do 1º lote a tabela está vazia e, depois dele, mostra 30 eventos, de 10:00:28 a 10:09:05, de 19 gêneros. A janela temporal é aplicada na seção 10, e os lotes seguintes chegam na seção 11.

```
%%capture
# Instalação do PySpark (só no Colab)
if NO_COLAB:
    %pip install -q "pyspark==4.0.3"
```

```
# Sessão do Spark
from pyspark.sql import SparkSession
from pyspark.sql import functions as F
from pyspark.sql.types import IntegerType, LongType, StringType, StructField, StructType, TimestampType

spark = (
    SparkSession.builder.master("local[2]")
    .appName("data-analytics — RA2")
    .config("spark.sql.session.timeZone", "America/Sao_Paulo")
    .config("spark.sql.shuffle.partitions", "2")
    .getOrCreate()
)
# Aplica o fuso também se já houver uma sessão aberta (reexecução da seção)
spark.conf.set("spark.sql.session.timeZone", "America/Sao_Paulo")
spark.sparkContext.setLogLevel("ERROR")

# Consultas que sobraram de uma execução anterior seguram arquivos das pastas abaixo
for consulta_antiga in spark.streams.active:
    consulta_antiga.stop()

print(f"Spark {spark.version} · fuso {spark.conf.get('spark.sql.session.timeZone')}")
```

```
# Pastas do streaming
import json
import shutil
from datetime import datetime, timedelta
from uuid import uuid4

import numpy as np

PASTA_ENTRADA = PASTA_STREAMING / "entrada"          # monitorada pelo Spark
PASTA_PUBLICACAO = PASTA_STREAMING / "publicacao"    # onde o lote é escrito antes de ser publicado
PASTA_CHECKPOINT = PASTA_STREAMING / "checkpoint"    # progresso das consultas
for pasta in (PASTA_ENTRADA, PASTA_PUBLICACAO, PASTA_CHECKPOINT):
    shutil.rmtree(pasta, ignore_errors=True)
    pasta.mkdir(parents=True)

print("Arquivos na entrada:", sorted(p.name for p in PASTA_ENTRADA.iterdir()))
```

```
# Faixas que alimentam os eventos
COLUNAS_FONTE = {"id_faixa": "id_faixa", "genero": "genero", "artista": "artista",
                 "duracao_s": "duracao_s", "rank": "rank"}

if "df_preparado" not in globals():
    arquivo_preparado = PASTA_PREPARADOS / "faixas_preparadas.csv"
    if arquivo_preparado.exists():
        df_preparado = pd.read_csv(arquivo_preparado)
if "df_preparado" in globals():
    faixas_fonte = (
        df_preparado[df_preparado["tipo_chart"] == "genero"]
        .rename(columns={v: k for k, v in COLUNAS_FONTE.items()})[list(COLUNAS_FONTE)]
    )
    origem_faixas = "conjunto preparado do RA1, charts de gênero"
elif "df_bruto" in globals():
    faixas_fonte = (
        df_bruto[df_bruto["chart_id_genero"] != 0]
        .rename(columns={"id": "id_faixa", "chart_nome_genero": "genero", "artist_name": "artista",
                         "duration": "duracao_s"})[list(COLUNAS_FONTE)]
    )
    origem_faixas = "df_bruto, charts de gênero (fonte provisória)"
else:
    raise FileNotFoundError(
        "Conjunto preparado não encontrado (df_preparado ou dados/preparados/faixas_preparadas.csv). "
        "Execute as seções do RA1 antes da seção 9."
    )
faixas_fonte = faixas_fonte.dropna().reset_index(drop=True)

print(f"Origem: {origem_faixas}")
```

```
print(f"{len(faixas_fonte)} pares faixa-gênero · {faixas_fonte['id_faixa'].nunique()} faixas · "
      f"{faixas_fonte['genero'].nunique()} gêneros · {faixas_fonte['artista'].nunique()} artistas")
```

```
# Esquema do evento de reprodução
esquema_eventos = StructType([
    StructField("id_evento", IntegerType()),        # sequencial, único na simulação
    StructField("id_faixa", LongType()),            # id da Deezer (passa do limite de um int)
    StructField("genero", StringType()),
    StructField("artista", StringType()),
    StructField("duracao_ouvida_s", IntegerType()), # segundos ouvidos, até a duração da faixa
    StructField("instante", TimestampType()),
])
FORMATO_INSTANTE = "%Y-%m-%d %H:%M:%S"   # o mesmo de timestampFormat na leitura
```

```
# Gerador de eventos
SEMENTE = 42
INICIO_SIMULACAO = datetime(2026, 10, 21, 10, 0)

sorteio = np.random.default_rng(SEMENTE)
proximo_id_evento = 1


def gerar_eventos(n, inicio):
    """Sorteia n reproduções das faixas de faixas_fonte, com instantes crescentes a partir de inicio.

    A chance de cada faixa segue o rank da Deezer (faixa mais popular, mais tocada). Entre um evento e o
    seguinte passam de 5 a 30 segundos; a duração ouvida vai de 30 segundos até a faixa inteira.
    """
    global proximo_id_evento
    pesos = faixas_fonte["rank"] / faixas_fonte["rank"].sum()
    escolhidas = faixas_fonte.iloc[sorteio.choice(len(faixas_fonte), size=n, p=pesos)]
    intervalos = np.cumsum(sorteio.integers(5, 31, size=n))
    lote = []
    for (_, faixa), segundos in zip(escolhidas.iterrows(), intervalos):
        duracao = int(faixa["duracao_s"])
        lote.append({
            "id_evento": proximo_id_evento,
            "id_faixa": int(faixa["id_faixa"]),
            "genero": faixa["genero"],
            "artista": faixa["artista"],
            "duracao_ouvida_s": int(sorteio.integers(min(30, duracao), duracao + 1)),
            "instante": (inicio + timedelta(seconds=int(segundos))).strftime(FORMATO_INSTANTE),
        })
        proximo_id_evento += 1
    return lote
```

```
# Publicação de um lote
lotes_publicados = 0


def publicar_lote(eventos):
    """Grava o lote em JSON (uma linha por evento) em PASTA_PUBLICACAO e move para PASTA_ENTRADA.

    O arquivo só aparece na entrada depois de escrito e fechado, então o Spark nunca lê um lote pela metade.
    """
    global lotes_publicados
    lotes_publicados += 1
    nome = f"lote_{lotes_publicados:03d}.json"
    arquivo = PASTA_PUBLICACAO / nome
    with arquivo.open("w", encoding="utf-8") as saida:
        for evento in eventos:
            saida.write(json.dumps(evento, ensure_ascii=False) + "\n")
    arquivo.replace(PASTA_ENTRADA / nome)
    print(f"Lote {lotes_publicados} publicado: {nome}, {len(eventos)} eventos, "
          f"de {eventos[0]['instante']} a {eventos[-1]['instante']}")
    return PASTA_ENTRADA / nome
```

```
# Leitura em streaming
eventos = (
    spark.readStream
    .schema(esquema_eventos)
    .option("timestampFormat", "yyyy-MM-dd HH:mm:ss")
    .json(str(PASTA_ENTRADA))
)
print("É um DataFrame em streaming?", eventos.isStreaming)
eventos.printSchema()
```

```
# Consulta de recebimento
TABELA_RECEBIMENTO = "recebimento_" + uuid4().hex
```

```
recebimento = (
    eventos.agg(
        F.count("*").alias("eventos_recebidos"),
        F.min("instante").alias("primeiro_instante"),
        F.max("instante").alias("ultimo_instante"),
    )
    .writeStream.format("memory")
    .queryName(TABELA_RECEBIMENTO)
    .outputMode("complete")
    .option("checkpointLocation", str(PASTA_CHECKPOINT / "recebimento"))
    .start()
)
print("Consulta ativa?", recebimento.isActive)
print("Arquivos na entrada:", sorted(p.name for p in PASTA_ENTRADA.iterdir()))
spark.table(TABELA_RECEBIMENTO).show(truncate=False)
```

```
# 1º lote
primeiro_lote = gerar_eventos(30, INICIO_SIMULACAO)
publicar_lote(primeiro_lote)
display(pd.DataFrame(primeiro_lote).head(10))
```

```
# Recebimento do 1º lote
recebimento.processAllAvailable()
spark.table(TABELA_RECEBIMENTO).show(truncate=False)
print("Eventos por gênero no 1º lote:")
display(pd.DataFrame(primeiro_lote)["genero"].value_counts().rename("eventos").to_frame())
```

```
# Encerramento da consulta de recebimento
recebimento.stop()
spark.catalog.dropTempView(TABELA_RECEBIMENTO)
print("Consulta de recebimento ativa?", recebimento.isActive)
```

10. Aplicação de janelas temporais

- Utilize uma janela **Tumbling** (intervalos sem sobreposição) ou **Sliding** (intervalos que podem se sobrepor).
- Informe o tipo e a duração da janela e o deslocamento, quando aplicável.
- Implemente uma agregação coerente com a análise, como contagem, soma ou média.
- Explique brevemente o que será calculado em cada intervalo.

O foco é demonstrar a compreensão do conceito; não é necessário justificar tecnicamente a duração escolhida.

Resposta do grupo:

Usamos uma única janela **tumbling de 5 minutos**, sem sobreposição. Cada intervalo inclui seu início e exclui seu fim: um evento às 10:05:00 entra em 10:05–10:10, e não em 10:00–10:05. O deslocamento é igual à duração (5 minutos), implícito nesse tipo de janela.

Contamos as reproduções por `genero` usando o instante do evento (`instante`), conforme o objetivo da seção 8. A tabela mostra a contagem de cada gênero em cada intervalo e permite identificar os gêneros com maior contagem, inclusive empates. As contagens representam eventos simulados, não audiência real da Deezer. O modo `complete` reexibe toda a tabela a cada lote; a saída `memory` permite mostrar os resultados na célula e no PDF.

```
# Agregação por gênero em janela tumbling de cinco minutos
from uuid import uuid4
from pyspark.sql import functions as F

if not eventos.isStreaming:
    raise ValueError("Execute a seção 9: eventos deve ser um DataFrame em streaming.")
# Interrompe consultas remanescentes antes de iniciar a consulta desta seção.
for consulta_anterior in spark.streams.active:
    consulta_anterior.stop()

agregado_janelas = eventos.groupBy(
    F.window("instante", "5 minutes"), "genero"
).agg(F.count("*").alias("reproducoes"))
```

```
# Consulta em memória com estado das janelas
# Nome único para poder rodar a seção novamente.
# O checkpoint é a pasta onde o Spark guarda o progresso desta consulta.
TABELA_JANELAS = "janelas_" + uuid4().hex
consulta = (
    agregado_janelas.writeStream
    .outputMode("complete")
    .format("memory")
    .queryName(TABELA_JANELAS)
```

```
    .option("checkpointLocation", str(PASTA_CHECKPOINT / TABELA_JANELAS))
    .start()
)
print(f"Consulta ativa: {consulta.isActive}; tabela em memória: {TABELA_JANELAS}")
```

```
# Tabela das janelas após o primeiro lote
consulta.processAllAvailable()
tabela_primeiro_lote = spark.table(TABELA_JANELAS)
quantidade_grupos = tabela_primeiro_lote.count()
# O limite é o número real de grupos, para que show() não esconda linhas além das primeiras 20.
tabela_primeiro_lote.select(
    F.col("window.start").alias("inicio"),
    F.col("window.end").alias("fim"), "genero", "reproducoes"
).orderBy("inicio", F.desc("reproducoes"), "genero").show(n=max(1, quantidade_grupos), truncate=False)

# Somamos as contagens da tabela para conferir quantos eventos foram recebidos.
total_reproducoes = tabela_primeiro_lote.agg(
    F.sum("reproducoes").alias("total")
).first()["total"]
print(f"Total de reproduções recebidas: {total_reproducoes or 0}")
# A consulta permanece ativa para os novos lotes e o encerramento da seção 11.
```

11. Resultados e interpretação

Apresente os resultados do processamento e demonstre como a análise se comporta com a chegada de novos registros.

```
# Preparação da comparação entre lotes
from datetime import datetime
from pyspark.sql import functions as F

if not consulta.isActive:
    raise RuntimeError("Execute as seções 9 e 10 antes de demonstrar os novos lotes.")


def ler_resultados_janelas():
    """Espera os arquivos disponíveis e copia o resultado atual para comparação."""
    consulta.processAllAvailable()
    return (
        spark.table(TABELA_JANELAS)
        .select(
            F.col("window.start").alias("inicio"),
            F.col("window.end").alias("fim"),
            "genero", "reproducoes",
        )
        .orderBy("inicio", F.desc("reproducoes"), "genero")
        .toPandas()
    )


resultados_atuais = ler_resultados_janelas()
lotes_demonstrados = [primeiro_lote]


def publicar_proximo_lote(quantidade=30):
    """Publica novas reproduções e mostra o resultado completo e as mudanças por janela."""
    global resultados_atuais
    if not consulta.isActive:
        raise RuntimeError("A consulta foi encerrada. Execute novamente as seções 9 e 10.")
    if not isinstance(quantidade, int) or quantidade <= 0:
        raise ValueError("A quantidade de eventos deve ser um inteiro positivo.")

    anteriores = resultados_atuais
    inicio = datetime.strptime(lotes_demonstrados[-1][-1]["instante"], FORMATO_INSTANTE)
    novo_lote = gerar_eventos(quantidade, inicio)
    publicar_lote(novo_lote)
    atuais = ler_resultados_janelas()

    chaves = ["inicio", "fim", "genero"]
    mudancas = atuais.merge(
        anteriores, on=chaves, how="outer", suffixes=("_depois", "_antes")
    ).fillna({"reproducoes_antes": 0, "reproducoes_depois": 0})
    for coluna in ("reproducoes_antes", "reproducoes_depois"):
        mudancas[coluna] = mudancas[coluna].astype("int64")
    mudancas["novas_reproducoes"] = mudancas["reproducoes_depois"] - mudancas["reproducoes_antes"]
    assert mudancas["novas_reproducoes"].ge(0).all(), "Uma contagem anterior diminuiu."
    assert int(mudancas["novas_reproducoes"].sum()) == len(novo_lote), "O lote não foi contado integralmente."

    lotes_demonstrados.append(novo_lote)
    total_esperado = sum(len(lote) for lote in lotes_demonstrados)
```

```
    total_recebido = int(atuais["reproducoes"].sum())
    assert total_recebido == total_esperado, "Total recebido diferente dos lotes publicados."

    resumo = mudancas.groupby(["inicio", "fim"], as_index=False)[
        ["reproducoes_antes", "novas_reproducoes", "reproducoes_depois"]
    ].sum()
    print(f"Após o lote {len(lotes_demonstrados)}: {total_recebido} reproduções recebidas.")
    with pd.option_context("display.max_rows", None):
        print("Contagens por gênero e janela:")
        display(atuais)
        print("Mudanças por janela (início incluído, fim excluído):")
        display(resumo)
    resultados_atuais = atuais
    return novo_lote
```

```
# Segundo lote: atualização de uma janela existente e abertura de novos intervalos
segundo_lote = publicar_proximo_lote()
resultados_apos_segundo_lote = resultados_atuais.copy()
```

```
# Terceiro lote: novas reproduções e comparação das contagens acumuladas
terceiro_lote = publicar_proximo_lote()
resultados_apos_terceiro_lote = resultados_atuais.copy()
```

Resposta do grupo:

As células “Segundo lote: atualização de uma janela existente e abertura de novos intervalos” e “Terceiro lote: novas reproduções e comparação das contagens acumuladas” publicam mais 30 eventos cada, em momentos distintos, e mostram a tabela após o processamento. Com os 30 eventos do primeiro lote, o total recebido passa de 30 para 60 e depois para 90.

O segundo lote acrescenta 3 reproduções em 10:05–10:10 (de 14 para 17), 18 em 10:10–10:15 e 9 em 10:15–10:20. O terceiro acrescenta 7 em 10:15–10:20 (de 9 para 16), 17 em 10:20–10:25 e 6 em 10:25–10:30. A janela 10:00–10:05 mantém suas 16 reproduções, pois nenhum evento dos dois novos lotes pertence a esse intervalo. Essas contagens são apresentadas nos resumos “Mudanças por janela” das duas células.

As janelas são atribuídas pelo instante de cada evento, não pelo horário em que o arquivo foi publicado. Assim, uma janela já exibida recebe novas contagens quando chegam eventos cujo instante ainda pertence a ela. A tabela por gênero permite comparar quais gêneros concentram as reproduções em cada intervalo; os valores representam uma simulação com semente fixa, e não a audiência real da Deezer.

Demonstração ao vivo: antes de executar a célula de encerramento, execute `publicar_proximo_lote(10)` em uma célula de código para publicar mais dez eventos e mostrar as mudanças. A função continua a sequência de IDs e instantes do lote anterior. Para repetir a demonstração desde o começo, execute novamente as seções 9, 10 e 11.

```
# Encerramento da consulta e limpeza das subpastas da simulação
import shutil

consulta.stop()
spark.catalog.dropTempView(TABELA_JANELAS)
for pasta in (PASTA_ENTRADA, PASTA_PUBLICACAO, PASTA_CHECKPOINT):
    destino = pasta.resolve()
    if destino.parent != (RAIZ / "dados" / "streaming").resolve():
        raise ValueError(f"Pasta fora do diretório da simulação: {destino}")
    if destino.exists():
        shutil.rmtree(destino)
print("Consulta ativa após o encerramento?", consulta.isActive)
print("Subpastas de entrada, publicação e checkpoint removidas; README preservado.")
```

12. Considerações do RA2

Responda brevemente:

- O que foi possível observar com o processamento em streaming?
- Qual foi a contribuição das janelas temporais para a análise?
- Essa abordagem faria sentido em uma aplicação real do projeto? Por quê?

Na última resposta, o grupo pode reconhecer que o streaming foi utilizado apenas como exercício didático, se esse for o caso.

Resposta do grupo:

[Preencha aqui.]